

Passport.js Session-Based Authentication — Complete Notes

Node.js · Express · PostgreSQL (pg) · express-session · passport-local

Source: Passport JS video series (intro → HTTP headers & cookies → express-session → passport-local implementation → how it all works). The original uses MongoDB/Mongoose — everything here is adapted to **PostgreSQL with the pg library** (connect-pg-simple session store, parameterized SQL queries, a **users** table instead of a Mongoose schema).

Sections and boxes marked **[+]** are additions beyond the videos — extra explanations, cookie security, and practical use-it-yourself material.

0. [+] The big picture — read this first

The whole system is easiest to hold in your head as **a club with a strict bouncer**:

- **Register** → your name goes on the member list. The club never writes down your actual password — only a scrambled *fingerprint* of it (the **hash**) plus the random sprinkle used while scrambling (the **salt**).
- **Login** → you say your password at the door **once**. The bouncer scrambles what you said the same way and compares fingerprints. Match → you get a **wristband** with a random number on it.
- The wristband says nothing about you. The bouncer's **notebook** maps wristband number → "member #42 is inside".
- **Every re-entry** → you just flash the wristband. The bouncer checks the notebook and knows exactly who you are. No password again.
- **Logout** → the notebook entry is erased. Your wristband now points to nothing.
- **Expiry** → the wristband ink fades after a day; you log in again.

Club world	Tech
Member list	users table
Password fingerprint + sprinkle	hash + salt columns
Bouncer comparing fingerprints	your verify callback

Club world	Tech
Wristband	the <code>connect.sid</code> cookie
Number on the wristband	the session ID
Bouncer's notebook	the <code>session</code> table (session store)
Notebook note "member #42 is inside"	<code>req.session.passport.user = 42</code>
Bouncer recognizing you each time	<code>deserializeUser</code> → <code>req.user</code>

Who does what — the four layers

The most confusing part of this stack is that four things share the work. Keep their jobs separate:

Layer	Its one job	Do you write it?
express-session	Cookies ↔ session rows: create sessions, set/read the signed cookie, load the data onto <code>req.session</code>	No — just configure
passport (core)	The framework: <code>authenticate()</code> , <code>initialize()</code> , <code>session()</code> , <code>req.login/logout/isAuthenticated</code> , the serialize/deserialize plumbing	No — just configure
passport-local	One strategy: "pull username + password out of <code>req.body</code> and hand them to a verify function"	No — just install
Your code	The actual decisions: find the user (SQL), check the password (pbkdf2/bcrypt), choose what goes into the session (the id)	Yes

One sentence to rule them all: *express-session gives every browser a persistent object on the server; passport stores "who's logged in" inside that object and re-reads it on every request; your verify callback is the only place a password is ever actually checked.*

1. The authentication landscape

Options ordered least → most complex. More complex ≠ better — it just means more moving parts to understand before you can implement it.

Option	Core idea	Typical use
Session-based (passport-local)	Server stores a session; browser holds a cookie carrying just the session ID	Server-rendered apps (EJS), classic websites, first projects
JWT (passport-jwt)	Signed token stored on the client; server just verifies the signature	SPAs (React), mobile apps, public APIs
OAuth 2.0	A <i>protocol</i> for delegated access ("Sign in with Google")	Consuming big third-party APIs, social login
Custom	Roll your own scheme	Rare — only with a strong reason

The two on the left are the standard choice for individuals/startups getting an app off the ground; OAuth comes later, when you have the time/resources or need third-party APIs.

Authentication vs authorization (interview classic)

- **Authentication** — *who are you?* Verifying identity (username + password). Session and JWT logins are about this.
- **Authorization** — *what may you access?* Permissions and scopes. **OAuth** is mostly this: an access token for the YouTube API carries **scopes** saying "can read analytics, cannot edit videos".

OAuth in one minute

- **OAuth is a protocol, not a product.** Auth0 and Okta are *companies* that sell that protocol as a service — the naming ("auth0" sounding like a protocol) is genuinely confusing.
- **OpenID Connect** is a closely related term — an identity layer built on top of OAuth.

- Overkill for a basic login system. Revisit when you integrate Google/GitHub-style APIs.
-

2. What Passport.js actually is

- Passport is an **Express middleware** that handles all the authentication logic for you.
- It is a *framework* with pluggable **strategies** — 500+ smaller middlewares written by independent developers that plug into the passport core: `passport-local`, `passport-jwt`, `passport-oauth2`, `passport-instagram`, ...
- You pick a strategy and hand it a **verify callback** containing *your* logic. Passport handles the wiring into Express routes.

Passport in one paragraph: on every request to a protected resource, passport asks the configured strategy — *is this user authenticated?* If yes → the request continues into the route handler. If no → passport returns **401 Unauthorized**.

Docs tip: the individual strategy pages (e.g. passport-local's own page) are thin and incomplete. The better docs for username/password auth live in the main passport documentation under *Username & Password* and *Configure* — and even those skip `serialize/deserialize`. Hence these notes.

3. HTTP headers (fast refresher)

- **HTTP is stateless** — the server forgets the client after every request/response cycle. This single fact is *why* cookies, sessions, and tokens exist.
 - Headers = metadata about a request or response. DevTools → Network shows three groups:
 - **General** — URL, HTTP method, status code
 - **Request headers** — client → server instructions: `Accept`, `Accept-Encoding`, `Accept-Language`, `User-Agent`, `Cookie`, ...
 - **Response headers** — server → client instructions: `Content-Type`, `Set-Cookie`, ...
 - Example handshake: the request says "I accept HTML/XML" (`Accept`) → the response answers "here's the data, and it's `text/html`" (`Content-Type`) so the browser knows how to display it.
 - MDN has the full catalogue. For authentication, only two headers matter: `Set-Cookie` (response) and `Cookie` (request).
-

4. Cookies — Set-Cookie and Cookie

The problem: stateless HTTP + no browser storage = the user re-enters credentials on *every refresh*. Any site doing that loses its users instantly.

The mechanism (memorize this loop):

1. Server includes `Set-Cookie: key=value; Expires=<date>` in a **response** header
2. Browser saves the cookie for that domain
3. Browser automatically attaches `Cookie: key=value` to **every subsequent request** to that domain
4. When `Expires` passes → browser deletes the cookie and stops sending it

Console demo — a cookie that lives 20 seconds:

```
const now = new Date();
```

```
now.setTime(now.getTime() + 20 * 1000);
```

```
document.cookie = `myCookie=hello; expires=${now.toUTCString()}`;
```

```
// Visible in DevTools → Application → Cookies. Gone after ~20s.
```

Why this powers auth: the server validates credentials **once**, responds with `Set-Cookie`, and **every future request automatically carries proof of "already logged in"**. The expiry is your "stay logged in for 2 weeks" dial.

Limits of cookies alone: ~4 KB, they live in the browser, and the user (or an attacker) can read them — so **never put credentials or secrets in a cookie**. That gap is exactly what server-side sessions fill.

5. Cookie vs session

	Cookie	Session
Lives on	Client (browser)	Server (session store = your DB)

	Cookie	Session
Size	~4 KB	As much as your DB allows
User can read it	Yes	No
OK for sensitive data	No	Yes
In this setup, holds	Only the session ID	The actual data (incl. who is logged in)

Mental model: **the cookie is the locker key; the session is the locker**. The server authenticates into the session using a secret key, so the client never holds anything sensitive — only an ID that is useless without the server.

[+] What does "signed" actually mean?

The cookie value isn't just the session id — it looks like `s:<sessionId>.<signature>` (URL-encoded as `s%3A...<signature>` in DevTools). The signature is a hash of the session id computed using the secret (an HMAC). On every request the server redoes the math: hash the id from the cookie with the secret, then compare it to the signature that arrived alongside. If someone edits the id in DevTools hoping to jump into another session, they can't produce a matching signature without the secret — mismatch → the cookie is treated as garbage → fresh anonymous session. So the client can *read* the cookie but cannot *forge* a valid one.

6. express-session

npm install express-session connect-pg-simple

The video uses `connect-mongo`; the PostgreSQL equivalent session store is `connect-pg-simple`.

```
// app.js
```

```
const session = require('express-session');
```

```
const pgSession = require('connect-pg-simple')(session);
```

```
const pool = require('./config/database');
```

```
app.use(
```

```

session({

  store: new pgSession({

    pool,           // your pg Pool

    tableName: 'session', // default name

    createTableIfMissing: true, // auto-create the table on startup

  }),

  secret: process.env.SECRET,

  resave: false,

  saveUninitialized: true,

  cookie: {

    maxAge: 1000 * 60 * 60 * 24, // 1 day = ms × sec × min × hr

  },

})

);

```

The table it manages (equivalent of the `sessions` collection in the video):

```

CREATE TABLE "session" (

  "sid"  varchar    NOT NULL PRIMARY KEY, -- session ID

  "sess" json       NOT NULL,           -- session data

  "expire" timestamp(6) NOT NULL        -- server-side expiry

);

```

The options

- **secret** — used to sign the session-ID cookie. If the secret is invalid, the session is invalid too. Changing it invalidates every existing session. Keep it in `.env`, never in git.
- **store** — where sessions persist. **Default = MemoryStore** (memory local to your app): fine for development, but it leaks memory and wipes on every restart — never use it in production. "Setting up a session store" is just a fancy way of saying "connect your database to the session middleware".
- **resave** — re-save unchanged sessions to the store on every request? Usually `false`.
- **saveUninitialized** — save brand-new sessions that were never modified? (`true` in the tutorial; many apps prefer `false`.) Both options control how the middleware reacts when nothing changed.
- **cookie.maxAge** — cookie lifetime in milliseconds (the `Expires` behaviour from section 4).

[+] Security flags to add in real projects:

cookie: {

maxAge: 1000 * 60 * 60 * 24,

httpOnly: true, // page JavaScript can't read the cookie → XSS cookie theft gets much harder (already true by default)

secure: true, // only sent over HTTPS → turn on in production (behind a proxy, also `app.set('trust proxy', 1)`)

sameSite: 'lax', // not attached to most cross-site requests → basic CSRF protection

},

These three concepts come back in the MDN *website security* article — same ideas, browser-side.

What the middleware does per request

1. **First visit:** creates a session row in the store, and responds with `Set-Cookie: connect.sid=<signed session id>` — `connect.sid` is express-session's default cookie name.
2. **Every later visit:** browser sends the cookie → middleware verifies the signature using `secret` → looks up the `sid` in the store → attaches the stored data to `req.session`.
3. **Expired or invalid cookie** → a fresh session is created.

You can verify the link yourself: the value inside the `connect.sid` cookie contains the same ID as the `sid` column in the `session` table.

`req.session` is a writable object — and it persists

```
app.get('/', (req, res) => {  
  
  if (req.session.viewCount) {  
  
    req.session.viewCount = req.session.viewCount + 1;  
  
  } else {  
  
    req.session.viewCount = 1;  
  
  }  
  
  res.send(`You have visited this page ${req.session.viewCount} times`);  
  
});
```

Anything you set on `req.session` inside any route is saved to the store automatically. Check it in `psql`:

```
SELECT sess FROM session;  
  
-- { "cookie": { ... }, "viewCount": 4 }
```

This is exactly the hook passport uses — it stashes its own data on the session object (you'll see `sess.passport.user` appear soon).

7. Project setup (PostgreSQL version)

project/

├─ app.js

├─ config/

| └─ database.js # pg Pool

```
|   └─ passport.js    # strategy + serialize/deserialize
|
|   └─ lib/
|
|   └─ passwordUtils.js # genPassword / validPassword
|
|   └─ routes/
|
|   └─ index.js
|
|   └─ authMiddleware.js # isAuth / isAdmin
|
└─ .env
```

`.env` (create it yourself — never committed):

`DB_STRING=postgresql://localhost:5432/tutorial_db`

`SECRET=some_long_random_string`

`config/database.js`:

```
const { Pool } = require('pg');
```

```
const pool = new Pool({
```

```
  connectionString: process.env.DB_STRING,
```

```
});
```

```
module.exports = pool;
```

The `users` table (replaces the Mongoose `UserSchema` with username/hash/salt):

```
CREATE TABLE users (
```

```
  id      SERIAL PRIMARY KEY,
```

```
  username VARCHAR(255) UNIQUE NOT NULL,
```

```
  hash    TEXT NOT NULL,
```

```
  salt    TEXT NOT NULL,
```

admin BOOLEAN DEFAULT false

);

app.js wiring — **order matters:**

```
require('dotenv').config();
```

```
const express = require('express');
```

```
const session = require('express-session');
```

```
const pgSession = require('connect-pg-simple')(session);
```

```
const passport = require('passport');
```

```
const pool = require('./config/database');
```

```
const routes = require('./routes');
```

```
const app = express();
```

```
// 1. Body parsing (built into Express now — no body-parser package needed)
```

```
app.use(express.json());
```

```
app.use(express.urlencoded({ extended: true }));
```

```
// 2. Session (must come BEFORE passport)
```

```
app.use(session({ /* config from section 6 */ }));
```

```
// 3. Passport — AFTER session, BEFORE routes
```

```
require('./config/passport');    // runs passport.use(strategy)
```

```
app.use(passport.initialize());
```

```
app.use(passport.session());
```

```
// 4. Routes
```

```
app.use(routes);
```

```
app.listen(3000);
```

- `passport.initialize()` — re-initializes/refreshes passport on every request so its state never goes stale.
 - `passport.session()` — the piece that reads the session and re-attaches the logged-in user on each request (full breakdown in section 11).
-

8. Password storage — salt & hash

Cardinal rule: never store a plaintext password. That's web-app 101 — the tricky part is the *how*.

Passport does not force any particular method on you. The standard approach has two phases:

Phase 1 — Registration (create): plaintext password + random **salt** → hash function → store { **salt**, **hash** } on the user row. The plaintext is received (unavoidable) but never persisted.

Phase 2 — Login (verify): look up the user by username → pull their stored **salt** and **hash** → run the *just-typed password* + the *stored salt* through the **same** hash function → compare the output with the stored hash. A hash function is deterministic — same inputs always give the same output — so a match proves the password is correct without the password ever being stored.

Why the salt?

- It's a pseudo-random per-user value mixed into the hash, so two users with the password **123** still get different hashes.
- Defeats precomputed lookup ("rainbow table") attacks.
- The salt is **not a secret** — it sits in the DB in plain sight. Its job is randomness, not secrecy.

```
// lib/passwordUtils.js
```

```
const crypto = require('crypto');
```

```
function genPassword(password) {
```

```
  const salt = crypto.randomBytes(32).toString('hex');
```

```

const genHash = crypto

  .pbkdf2Sync(password, salt, 10000, 64, 'sha512')

  .toString('hex');

return { salt, hash: genHash };

}

```

```

function validPassword(password, hash, salt) {

  const hashVerify = crypto

    .pbkdf2Sync(password, salt, 10000, 64, 'sha512')

    .toString('hex');

  return hash === hashVerify;

}

```

```

module.exports = { genPassword, validPassword };

```

pbkdf2Sync(password, salt, iterations, keylen, digest):

- **10000** iterations — the IETF spec (RFC 8018, password-based cryptography) recommends at least 1,000; ~10 million for very high security. Iterations make the hash deliberately **slow**, which is the whole point: brute-forcing becomes expensive.
- **64** — output length in bytes; **'sha512'** — the digest algorithm; **.toString('hex')** — a storable string.
- **PBKDF2 is an IETF standard** — not a random function choice, but the documented "this is how you should do it".

TOP note: The Odin Project's auth lessons use **bcryptjs** instead. Same concept, simpler API — and bcrypt embeds the salt *inside* the hash string, so you store one column instead of two:

```

const bcrypt = require('bcryptjs');

const hash = await bcrypt.hash(password, 10);      // register

const ok = await bcrypt.compare(password, user.hash); // login

```

Either is interview-acceptable. The concept being tested is *slow, salted, one-way hashing*.

9. Configuring the passport-local strategy

```
npm install passport passport-local
```

```
// config/passport.js
```

```
const passport = require('passport');
```

```
const LocalStrategy = require('passport-local').Strategy;
```

```
const pool = require('./database');
```

```
const { validPassword } = require('../lib/passwordUtils');
```

```
// Only needed when your form fields are NOT named "username" and "password"
```

```
const customFields = {
```

```
  usernameField: 'uname',
```

```
  passwordField: 'pw',
```

```
};
```

```
const verifyCallback = async (username, password, done) => {
```

```
  try {
```

```
    const { rows } = await pool.query(
```

```
      'SELECT * FROM users WHERE username = $1',
```

```
      [username]
```

```
    );
```

```
    const user = rows[0];
```

```

if (!user) return done(null, false); // no such user → reject

const isValid = validPassword(password, user.hash, user.salt);

if (isValid) return done(null, user); // success

return done(null, false);          // wrong password → reject

} catch (err) {

  return done(err);                // DB / server error

}

};

passport.use(new LocalStrategy(customFields, verifyCallback));

```

How the parameters get populated

On `POST /login`, passport inspects `req.body` for the configured field names, grabs their **values**, and calls your verify callback with them. The HTML form's `name` attributes are what become the `req.body` keys — so `<input name="uname">` and `<input name="pw">` must match `usernameField/passwordField` (defaults: `username, password`). If they don't match, passport can't find the credentials, authentication fails ("Missing credentials"), and your callback never even runs.

The `done( )` contract — the only thing passport cares about

Call	Meaning	Effect
<code>done(null, user)</code>	Authenticated	User is serialized into the session; request proceeds
<code>done(null, false)</code>	Bad credentials	401 Unauthorized / <code>failureRedirect</code>
<code>done(err)</code>	Something broke	Passed to Express error handling

Key insight: passport does **not** dictate your database or verification logic. The docs show Mongo, we use Postgres — irrelevant. Any DB, any hashing library, any custom rules. The only contract is what you pass to **done**.

serializeUser / deserializeUser

```
passport.serializeUser((user, done) => {

  done(null, user.id); // what gets stored IN the session

});

passport.deserializeUser(async (id, done) => {

  try {

    const { rows } = await pool.query(

      'SELECT * FROM users WHERE id = $1',

      [id]

    );

    done(null, rows[0]); // becomes req.user

  } catch (err) {

    done(err);

  }

});
```

- **serialize** = user → session. Runs **once, at login**. Stores just the id at **req.session.passport.user**.
 - **deserialize** = session → user. **Runs on every request while logged in**. Takes the stored id, looks the user up in the DB, attaches the full row as **req.user**.
 - Why only the id? The session stays tiny, and the user data is always fresh from the DB on every request.
-

10. Register & login routes

```
// routes/index.js

const router = require('express').Router();

const passport = require('passport');

const pool = require('../config/database');

const { genPassword } = require('../lib/passwordUtils');

// REGISTER — phase 1 (create)

router.post('/register', async (req, res, next) => {

  try {

    const { salt, hash } = genPassword(req.body.pw);

    await pool.query(

      'INSERT INTO users (username, hash, salt) VALUES ($1, $2, $3)',

      [req.body.uname, hash, salt]

    );

    res.redirect('/login');

  } catch (err) {

    next(err);

  }

});

// LOGIN — phase 2 (verify): passport does the work

router.post(
```

```

'/login',

passport.authenticate('local', {

  successRedirect: '/login-success',

  failureRedirect: '/login-failure',

})

);

```

What `passport.authenticate('local', options)` does, step by step:

1. Finds the strategy registered under the name `'local'`
2. Extracts the credentials from `req.body` using the configured field names
3. Runs your **verify callback**
4. `done(null, user) → internally calls req.login(user) → serializeUser → user id written into the session → successRedirect`
5. `done(null, false) → failureRedirect` (or a plain 401 if no redirect was configured)

Because both outcomes are handled by the redirect options, the route needs **no handler function** after the middleware.

[+] Auto-login after register

Redirecting a fresh user to `/login` to retype the password they entered two seconds ago is clunky. `req.login` (the same function `passport.authenticate` calls internally) fixes it:

```

router.post('/register', async (req, res, next) => {

  try {

    const { salt, hash } = genPassword(req.body.pw);

    const { rows } = await pool.query(

      'INSERT INTO users (username, hash, salt) VALUES ($1, $2, $3) RETURNING *',

      [req.body.uname, hash, salt]

    );

  }

```

```

req.login(rows[0], (err) => { // serialize straight into the session

  if (err) return next(err);

  res.redirect('/');

});

} catch (err) {

  next(err);

}

});

```

(**RETURNING *** is the pg way to get the inserted row back in the same query — you need the row, with its **id**, to hand to **req.login**.)

11. What actually happens on every request (the deep dive)

Drop a debug middleware in **app.js** (after **passport.session()**) and watch:

```

app.use((req, res, next) => {

  console.log(req.session);

  console.log(req.user);

  next();

});

```

State	req.session	req.user
Before login	{ cookie: { ... } }	undefined

State	req.session	req.user
After login	<code>{ cookie: { ... }, passport: { user: 42 } }</code>	full user row
After logout	<code>passport.user</code> removed	<code>undefined</code>

That `passport: { user: 42 }` was written by **serializeUser** during `passport.authenticate` — the 42 is the same `id` you'd see with `SELECT id FROM users`.

`passport.initialize()` and `passport.session()` run on **every** route request. Conceptually, `passport.session()` does this:

// pseudocode of what runs on each request

```
if (req.session.passport && req.session.passport.user != null) {

  const user = await deserializeUser(req.session.passport.user); // DB lookup

  req.user = user;

}

// else: nobody is logged in → req.user stays undefined
```

So "logged in" literally means: **a user id is sitting inside the session**. Passport cleverly piggybacks on express-session's persistence mechanism — that's the whole trick.

[+] What an authenticated request costs: signature check (cheap) + one `SELECT` for the session row + one `SELECT` for the user row. Two DB reads on *every* protected request. That's the price of easy revocation — and exactly the cost JWT removes later (the token itself carries the proof, so the server looks nothing up). Keep this framing in mind for the JWT part of the series.

Full request lifecycle

Browser

Server

| POST /login (uname, pw) |

|----->| express-session: load/create session (PG store)

```

|                                     | passport.authenticate('local')
|
|                                     |   verify cb: SELECT user → validPassword()
|
|                                     |   done(null, user) → serializeUser
|
|                                     |   req.session.passport.user = user.id → saved to PG
|
| 302 redirect + Set-Cookie sid      |
|
|<-----|
| GET /protected (Cookie: sid)      |
|
|----->| session loaded from store → passport.session()
|
|                                     |   deserializeUser(id) → req.user = full user
|
|                                     |   isAuthenticated → route handler runs

```

12. Built-in request methods (added by passport)

- **req.login(user, cb)** — logs a user in (serialize + write to session). Called automatically inside **passport.authenticate**, so you rarely call it yourself — the main manual use is auto-login right after registration.
- **req.logout(cb)** — deletes **req.session.passport.user**. On the next request, **passport.session()** finds no id → user is logged out.
- **req.isAuthenticated()** — returns **true** iff a serialized user exists in the session. The one you'll use constantly.
- **req.isUnauthenticated()** — the opposite; rarely needed.

⚠ Version trap: since **Passport 0.6**, **req.logout** is asynchronous and **requires a callback**:

```

router.get('/logout', (req, res, next) => {

  req.logout((err) => {

    if (err) return next(err);
  });
});

```

```
    res.redirect('/');

  });

});
```

Older tutorials (this video series included) call `req.logout()` bare — on current passport that throws an error.

13. Protecting routes — custom auth middleware

Instead of hand-checking `req.isAuthenticated()` inside every route, wrap the check in reusable middleware: pass → `next()`, fail → error status. Then gate any route by dropping the middleware in front of the handler.

```
// routes/authMiddleware.js
```

```
module.exports.isAuth = (req, res, next) => {
```

```
  if (req.isAuthenticated()) return next();
```

```
  res
```

```
    .status(401)
```

```
    .json({ msg: 'You are not authorized to view this resource' });
```

```
};
```

```
module.exports.isAdmin = (req, res, next) => {
```

```
  if (req.isAuthenticated() && req.user.admin) return next();
```

```
  res
```

```
    .status(403)
```

```
    .json({ msg: 'You are not authorized to view this resource because you are not an admin' });
```

```
};
```

Usage:

```
const { isAuth, isAdmin } = require('./authMiddleware');
```

```
router.get('/protected', isAuth, (req, res) => {
```

```
  res.send('You made it to the route');
```

```
});
```

```
router.get('/admin', isAuth, isAdmin, (req, res) => {
```

```
  res.send('You made it to the admin route');
```

```
});
```

You can chain `isAuth`, `isAdmin`, or let `isAdmin` do both checks itself (as above) — both patterns work.

Admin flag in Postgres:

-- if the column wasn't created earlier:

```
ALTER TABLE users ADD COLUMN admin BOOLEAN DEFAULT false;
```

-- register someone as an admin:

```
INSERT INTO users (username, hash, salt, admin) VALUES ($1, $2, $3, true);
```

Status-code nuance (interviewers like this): 401 Unauthorized = not authenticated (not logged in / bad credentials). **403 Forbidden** = authenticated but lacking permission. The video returns 401 in both middlewares; strictly, a logged-in non-admin should get **403** — which is what the code above does.

The bigger idea: these middleware gates are freely composable. Role checks, resource-ownership checks, whatever your app needs — you're not boxed in by the framework.

14. Common traps

1. **Middleware order.** `express.json/urlencoded` → `session()` → `passport.initialize()` → `passport.session()` → routes. Break the order and `req.session` / `req.user` won't exist when they're needed.
 2. **Field-name mismatch.** Form input `names` must match `usernameField/passwordField` (defaults `username/password`), or the `verify` callback never runs and login "mysteriously" fails with missing credentials.
 3. **MemoryStore in production.** The default store leaks memory and loses every session on restart. Use `connect-pg-simple`.
 4. **Rotating or leaking the `secret`.** Changing it invalidates every session signature (all users logged out). Keep it in `.env`, out of git.
 5. **`done(null, false)` vs `done(err)`.** Wrong credentials are *not* an error — reject cleanly with `false`. Reserve `done(err)` for genuine failures (DB down, exception thrown).
 6. **`req.logout()` without a callback** throws on Passport ≥ 0.6.
 7. **PBKDF2 parameters must match exactly** in `genPassword` and `validPassword` (iterations, keylen, digest) — otherwise no password will ever validate.
 8. **Serializing the whole user object.** It works, but bloats the session and goes stale. Store the id; deserialize fetches fresh data.
 9. **SQL injection.** Always use parameterized queries (`$1`, `$2`); never template-string user input into SQL.
 10. **Trusting the cookie's contents.** `connect.sid` holds only a *signed* session id — the client can read it but can't forge a valid one without the server's secret. Still: no sensitive data in cookies, ever.
-

15. [+] Use it in your own project — the recipe

Bolting this onto any Express + Postgres app:

1. `npm install express express-session connect-pg-simple passport passport-local pg dotenv`
2. Create the database, the `users` table, and a `.env` with `DB_STRING` + `SECRET` (§7)
3. `config/database.js` — export a `Pool` (§7)
4. `lib/passwordUtils.js` — `genPassword` / `validPassword` (§8) — or `bcrypt`, same shape
5. `config/passport.js` — `verify` callback, `passport.use(...)`, `serialize/deserialize` (§9)

6. `app.js` in this exact order: parsers → `session()` with the pg store → `require('./config/passport')` → `passport.initialize()` → `passport.session()` → routes (§7)
7. Routes: `POST /register` (hash + `INSERT`, optionally `req.login`), `POST /login` (`passport.authenticate('local', {...})`), `GET /logout` (`req.logout(cb)`) (§10, §12)
8. Drop `isAuth` / `isAdmin` in front of anything private (§13)

The forms (plain HTML or EJS) — only two rules: the form's `action` + `method` must match your POST route, and each input's **name attribute** must match the strategy's field names:

```
<form action="/login" method="POST">

  <input name="uname" placeholder="username" />

  <input name="pw" type="password" placeholder="password" />

  <button type="submit">Log in</button>

</form>
```

Using the defaults instead (no `customFields` object)? Then `name="username"` and `name="password"` — that's what TOP does.

Sanity checks while building: DevTools → Application → Cookies (is `connect.sid` there?), `SELECT sess FROM session;` in psql (does `passport.user` appear in the JSON after login?), and the §11 debug middleware printing `req.session` / `req.user`.

16. [+] Debugging — symptom → cause

Symptom	Most likely cause
Login always fails and the verify callback never runs	Form <code>names</code> don't match <code>usernameField/passwordField</code> → "Missing credentials" (§9)
<code>req.user</code> is always <code>undefined</code>	Middleware order wrong, <code>passport.session()</code> missing, or <code>serialize/deserialize</code> not defined (§7, §9)

Symptom	Most likely cause
Correct password still rejected	<code>genPassword/validPassword</code> params differ (iterations/keylen/digest), or hash and salt passed in the wrong order (§8)
Everyone logged out when the server restarts	Still on the default MemoryStore (§6)
Everyone logged out at once, no restart	<code>SECRET</code> changed → every cookie signature invalid (§6)
<code>Error: req#logout requires a callback function</code>	Passport ≥ 0.6 — pass the callback (§12)
Login succeeds but the very next request gets 401 (React dev server)	Cross-origin cookies: the frontend must fetch with <code>credentials: 'include'</code> and the API needs <code>cors({ origin: <frontend URL>, credentials: true })</code> . Cookies don't cross origins by default — you <i>will</i> hit this in MERN.
<code>session</code> table keeps old rows around	Normal — connect-pg-simple deletes expired rows periodically

17. [+] FAQ — questions everyone has

Is my password sent on every request? No. It travels exactly once, at login. After that only the cookie — a meaningless id — travels. And it's never *stored* anywhere, only its salted hash.

So the server re-checks my password on every page? No — once, in the verify callback. Afterwards the session id *is* the proof. That is the entire point of the system.

If someone steals my cookie, are they me? Until it expires, yes — that's *session hijacking*. It's why `httpOnly` (scripts can't read it), `secure` + HTTPS (can't be sniffed), and a sane `maxAge` matter (§6).

Can the server force-logout someone? Yes — delete their row from the `session` table and their cookie points at nothing. Instant revocation is the big advantage sessions have over JWTs.

Can I keep other things in the session? Yes — it's just an object (`viewCount`, a cart, preferences). Passport only claims one corner of it: `req.session.passport.user`.

Why did the video use `uname/pw`? Purely to demonstrate `customFields`. The defaults (`username/password`) are fine — use them and skip the `customFields` object entirely.

Session or JWT for my projects? Server-rendered EJS app → this setup. Separate React frontend + Express API → usually JWT (next in the series), though sessions do work cross-origin with the CORS/credentials setup from §16.

18. Interview Q&A

Q1. Authentication vs authorization? Authentication verifies *who you are* (login). Authorization decides *what you may do* (permissions/scopes). Session/JWT login = authentication; admin checks and OAuth scopes = authorization.

Q2. HTTP is stateless — what does that mean for login? The server forgets the client after each request/response cycle. Without cookies/sessions/tokens, the user would re-enter credentials on every page load. These mechanisms re-identify the client on each request.

Q3. How do `Set-Cookie` and `Cookie` work together? The server sends `Set-Cookie` in a response; the browser stores it and then automatically attaches it as the `Cookie` request header on every future request to that domain, until it expires.

Q4. Cookie vs session? A cookie is small, client-side, and user-readable. A session is server-side data keyed by an id and can safely hold sensitive information. In session auth the cookie carries only the session id; the data lives on the server.

Q5. What's inside the `connect.sid` cookie? A session id signed with the session `secret` — no user data. The server uses the id to look up the session in the store; the signature prevents tampering.

Q6. What is a session store, and why replace the default? It's where session data persists. The default `MemoryStore` lives in application RAM — it leaks memory and wipes on restart — so production apps plug in a database-backed store (`connect-pg-simple` for Postgres).

Q7. What does `passport.authenticate('local')` actually do? It's middleware that finds the strategy named `'local'`, pulls the credentials out of `req.body`, runs your verify callback, and — based on what you pass to `done` — either logs the user in (serializes them into the session) or fails with a 401/redirect.

Q8. What's the verify callback's contract? You may verify however you like — any database, any hashing. You must call `done(err)` on server errors, `done(null, false)` for bad

credentials, and `done(null, user)` on success. That return contract is all passport cares about.

Q9. `serializeUser` vs `deserializeUser`? `serializeUser` runs at login and decides what gets stored in the session — usually just the user id. `deserializeUser` runs on every subsequent request, turning that id back into a full user via a DB lookup and attaching it to `req.user`.

Q10. Why store only the user id in the session? It keeps the session payload tiny, and the user data stays fresh — any change to the user row shows up on the very next request.

Q11. Why salt passwords? The salt is a random per-user value mixed into the hash, so identical passwords produce different hashes and precomputed rainbow-table attacks fail. It's stored openly — its job is uniqueness, not secrecy.

Q12. Why PBKDF2/bcrypt instead of a plain SHA-256 hash? They are deliberately slow (thousands of iterations / a cost factor), making brute-force attacks expensive. Fast general-purpose hashes can be brute-forced at enormous speed.

Q13. What do `passport.initialize()` and `passport.session()` do? `initialize` sets passport up freshly on each request. `session` checks `req.session.passport.user`; if an id is present it deserializes it and populates `req.user`. Together they keep login state alive across requests.

Q14. How does `req.isAuthenticated()` work under the hood? It's essentially a check that a serialized user exists on the session (`req.session.passport.user` is set) — which only ever happens after a successful `passport.authenticate` login.

Q15. Sessions vs JWT — when would you pick each? Sessions: server-rendered apps, state lives on the server, easy revocation (delete the session row). JWT: SPAs, mobile apps, and APIs — stateless and scales horizontally, but revocation is harder because the token lives on the client until it expires.

Q16. 401 vs 403? 401 Unauthorized: not authenticated (no or invalid login). 403 Forbidden: authenticated but not permitted — e.g. a logged-in non-admin hitting an admin route.

Q17. How would you secure the session cookie? `httpOnly` so page scripts can't read it, `secure` so it only travels over HTTPS, `sameSite` to stop it riding along on cross-site requests, a reasonable `maxAge`, and a strong `secret` kept in `.env` and out of the repo.

Next in the series: the passport-JWT strategy — token-based auth for SPAs and APIs (React frontends). The session mechanics above are the foundation; JWT swaps the "server remembers you" model for a "you carry signed proof" model.